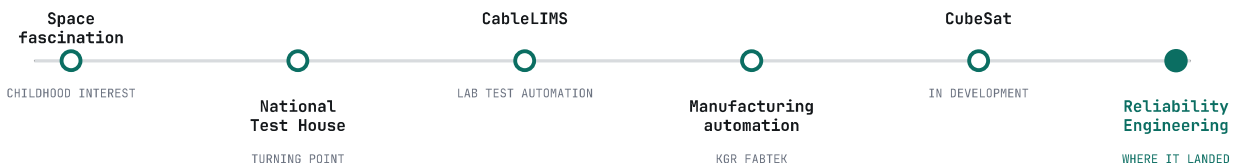


Siddharth Hariharan

Quality Engineering Portfolio · Aerospace & Manufacturing

When I first got interested in aircraft, I mostly thought engineering ended once something worked. If it flew, if it held together, if the number came out right, that felt like the finish line. A summer at India's National Test House changed my mind about that. Engineering there did not stop when a result came in. Someone had to write down exactly how the test was run. Someone had to check that result against a written standard. And someone else, later, had to be able to read the record and reach the same conclusion without asking you anything. That is closer to what good engineering actually means: documentation, repeatable procedure, standards, and a record you can trace back to its source. Every project on the next few pages grew out of some version of that.

HOW THE INTEREST MOVED



READING THIS PORTFOLIO

Two systems built and in real use, and one flight computer in active development. Each page follows the same shape: the problem, how it was approached, and the decisions behind it. None of them are feature lists.

WHAT TO EXPECT

Every diagram is drawn from the actual system design, and every screenshot is from the running system. If a claim could not be backed by the real thing, it did not make it onto the page.

CableLIMS

Quality System National Test House (SR) · 2026 FastAPI SQLAlchemy React + TypeScript
JWT Auth

PROBLEM

Cable test results at NTH were checked against IS standards by hand, and certificates were typeset manually after every test run.

WHY IT MATTERED

Hours spent on arithmetic were hours taken from the judgment calls a written standard cannot make for you, and from the documentation trail that has to exist afterward so someone else can rely on the result.

APPROACH

Keep the test engineer in charge of the test. Automate only the parts of the workflow that have exactly one correct answer, and document the rest.

ENGINEERING DECISIONS

- Calculation engine mirrors IS pass/fail logic exactly, with no interpretation layer between the standard and the result. The standard says what it says.
- Role-based views separate what a scientist can approve from what an assistant can log.
- Certificates carry a QR code back to the source record, not just a static PDF, so the result stays traceable after the fact.

ROOT CAUSE EXAMPLE

Deployment stalled on a lab machine running Python 3.14 as default.
pip attempted to build pydantic-core & Pillow from source
(SOABI tag: cp314-win_amd64. No prebuilt wheel yet for 3.14).

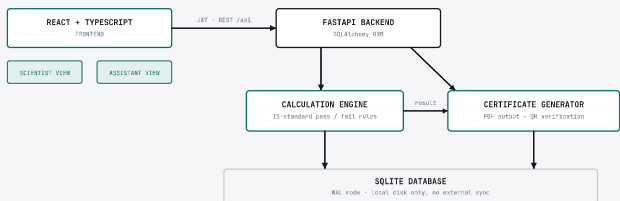
Fix: rebuilt the virtual environment against Python 3.12
instead of chasing the build toolchain.
Root cause: dependency availability, not application code.

DATA INTEGRITY REVIEW

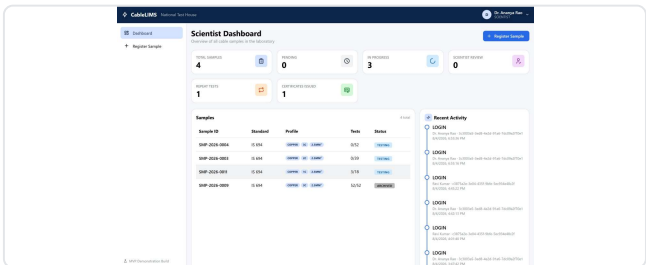
Ran a full outbound-call audit before this touched real lab data: no telemetry, no analytics SDK, no external URLs. The only outbound contact is a one-time dependency install; at runtime the app never leaves the lab's local network.

LESSONS LEARNED

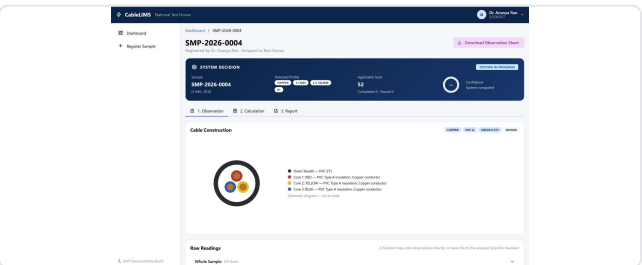
A calculation engine only earns trust if it is exactly as strict as the standard it is checking against: no looser, no stricter.



SYSTEM ARCHITECTURE · REQUEST, CALCULATION, AND CERTIFICATE FLOW



SCIENTIST DASHBOARD · LIVE SAMPLE QUEUE



SAMPLE DETAIL · SYSTEM-COMPUTED DECISION

Manufacturing Quotation Automation

Process Automation

KGR Fabtek • 2026

Python

Streamlit

Automated Data Extraction

PROBLEM

Every sheet-metal fabrication job started with an engineer manually reading a drawing and pricing it against a rate card that moves with material and process cost.

ENGINEERING WORKFLOW

- Extract title-block and geometry fields from the fabrication drawing (PDF).
- Compute per-part cost across laser, shear, and bend processes from a configurable rate card.
- Route every extracted field through a human review step before it reaches a cost.
- Generate the quotation workbook in the client's existing format.

INTERESTING ENGINEERING DECISION

Rates are stored as a configurable card, not hardcoded, because material and process cost move independently and often. The pricing logic shouldn't need a code change every time a supplier updates a number.

WHY THIS IS A PROCESS PROJECT, NOT AN AI PROJECT

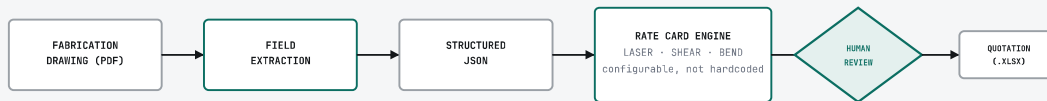
The drawing-reading step is automated. The pricing decision is not. The rate card engine, the review gate, and the output format are all deterministic and do not care how the drawing got read. This is a process-standardization project with one automated step inside it, not the other way around.

HUMAN VERIFICATION

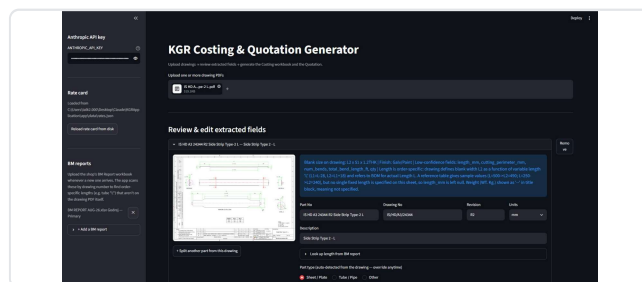
Every extracted field is reviewed before it reaches a cost calculation. A wrong number at this stage does not just cost time. On a real quotation, it costs a client relationship.

LESSONS LEARNED

Automation is only valuable if the person at the other end still trusts the output enough to act on it. Speed that is not trusted does not get used, and speed that is not used is just a demo.



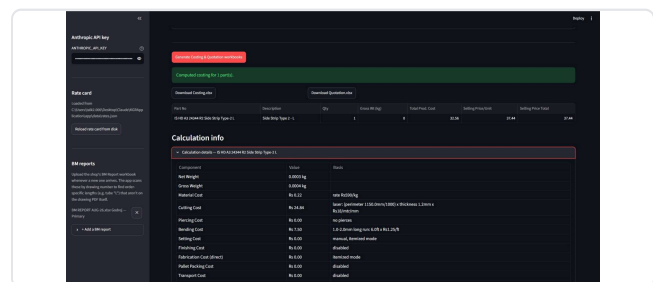
DATA FLOW • DRAWING TO QUOTATION, WITH A MANDATORY HUMAN REVIEW GATE



FIELD REVIEW • HUMAN VERIFICATION GATE

PLANNED NEXT PHASES

A batch-processing app for multi-drawing jobs, plus a second costing format (Antune). The rates move; the logic should not.



COSTING OUTPUT • TRACEABLE PER-PROCESS BREAKDOWN

WHAT THIS PROJECT IS REALLY ABOUT

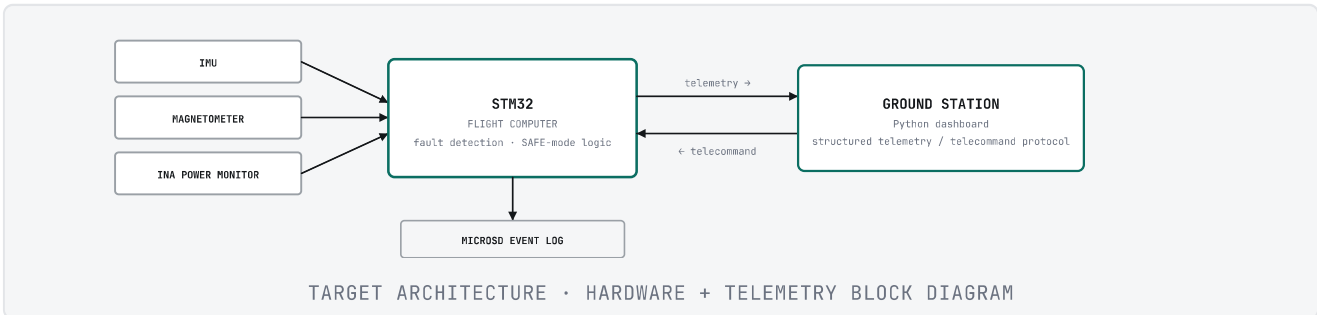
Not the extraction model. The real work was deciding which steps have exactly one correct answer, and which still need an engineer.

CubeSat Flight Computer & Verification Platform

IN ACTIVE DEVELOPMENT 2026 – Present STM32 • CubeIDE / HAL Embedded C Python ground station

MISSION OBJECTIVE

Define requirements and system architecture for a CubeSat-class flight computer, then build the verification plan alongside the hardware rather than after it: telemetry validation, subsystem health monitoring, SAFE-mode recovery, and fault tolerance, each written down before the corresponding hardware exists. Documentation and design assurance are being developed at the same pace as the build itself, not added on at the end.



VERIFICATION STRATEGY (PLANNED METHODOLOGY)

The development plan is a staged campaign, each stage meant to remove a different kind of doubt before the next one starts: software simulation → hardware bring-up → subsystem verification → telemetry validation → SAFE-mode verification → fault-injection testing → system integration. Simulation checks the logic. Bring-up checks that the hardware agrees with the logic. Fault injection, breaking the system on purpose, checks whether it can still tell you the truth about itself when something goes wrong. That is the stage I have put the most thought into documenting properly, since it is the one most likely to get skipped if the paperwork only happens after the fact.

DOCUMENTATION AND DESIGN ASSURANCE

Requirements, interface definitions, and the verification plan itself are being written as the system is designed, not reconstructed afterward to match what got built. If a reviewer asked for the reasoning behind the SAFE-mode logic today, there is already a document that answers that, even though hardware bring-up is not finished.

PROJECT STATUS

COMPLETED

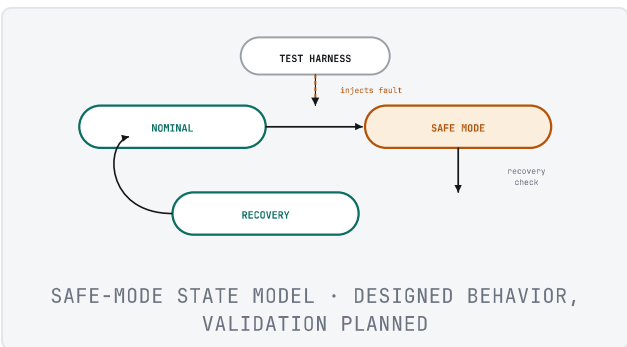
- System architecture
- Flight software design
- Verification methodology
- Ground-station architecture

IN PROGRESS

- STM32 hardware bring-up
- Sensor integration
- Telemetry implementation
- Ground-station software

PLANNED

- SAFE-mode validation
- Fault-injection testing
- Integrated subsystem testing
- End-to-end verification



WHY VERIFICATION PLANNING HAPPENS NOW

It would be easy to write the verification plan after the hardware works, as a formality. I am doing it the other way around, because a flight computer that has only ever been asked to succeed has not actually been tested yet. It has just been hoped that it would work, and hope is not something I want to put in a design review.

What these projects taught me

Not a project list. The pattern underneath all four.

QUALITY

A system is not good because it works. It is good because you can show, after the fact, that it was checked properly.

REPEATABILITY

A procedure only counts as engineering once someone else can run it on a different day and land on the same answer.

DOCUMENTATION

If the reasoning behind a decision only lives in my head, the decision is not finished yet.

TRACEABILITY

Every result should point back to where it came from, the standard it was checked against, and who checked it.

CONTINUOUS IMPROVEMENT

None of these systems were right the first time. CableLIMS did not have QR verification until I noticed how easy it was to lose a certificate's origin. The improvement usually came from noticing a small gap, not from a big redesign.

ENGINEERING JUDGMENT

Automation is only worth having if it leaves room for someone to disagree with it. Every system here has a place where a person can stop the process and say that does not look right.

WHAT I AM LOOKING FOR NOW

I did not set out to become interested in quality and documentation. Every project in this portfolio started as an attempt to build something, and somewhere in each one I ended up spending more time on how you would prove it was built right than on the building itself. I like environments where that gets taken seriously. Where a procedure exists because someone thought carefully about what could go wrong. Where a record is expected to hold up if someone questions it later. And where getting something right the first time matters less than being able to show your work. That is the kind of engineering team I am looking to join next.

I still like building things. What changed is what I consider the job finished. It used to be finished when something worked. Now it is finished when someone else could pick up my documentation and reach the same conclusion I did, without needing to ask me a single question.